Independent University, Bangladesh

CSE313 · Practice problem 02 (LL(1) Grammar) · Google Classroom

Q3 (Operator Precedence Parser) is a final-exam topic, so it is not solved here.

Question paper

1. Consider the grammar G with start symbol S, non-terminals {S, A, B}, terminals {a, b, c, d}. (a) Find FIRST(S), FIRST(A), FIRST(B). (b) Find FOLLOW(S), FOLLOW(A), FOLLOW(B). (c) Construct the parsing table, paying special attention to the ε-production. (d) Parse aaabbc. (e) Parse bbc.

```
S → a A b
A → c B
B → d | ε
```

2. Consider the grammar G with start symbol S, non-terminals {S, A, B}, terminals {a, b, c}. (a) FIRST sets. (b) FOLLOW sets. (c) Parsing table. (d) Parse acdb.

```
S → A B
A → a A | ε
B → b B | c
```

Answers (exam layout, with comments)

Answer 1

WRITE THIS

Numbered rules: 1. S → a A b 2. A → c B 3. B → d 4. B → ε

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
S	{ a }	{ $ }
A	{ c }	{ b }
B	{ d, ε }	{ b }

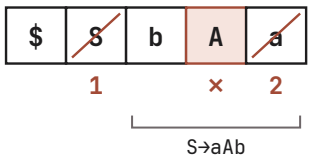
Step 2 · LL(1) parsing table

	a	b	c	d	$
S	S→aAb				
A			A→cB		
B		B→ε		B→d	

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3a · Parse `a a a b b c` (horizontal stack)

Buffer: `a a a b b c $` LA ↑ starts at `a`



How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

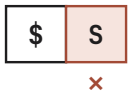
1. pop S (LA = a): S→aAb, push b A a
2. match a, pop it, move LA → a

err. Top = A, LA = a: cell M[A, a] is empty → error

Parsing is not possible. Reason: Top = A, LA = a: cell M[A, a] is empty → error. ∴ error, the string `a a a b b c` is not in the language.

Step 3b · Parse `b b c` (horizontal stack)

Buffer: `b b c $` LA ↑ starts at `b`



How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

err. Top = S, LA = b: cell M[S, b] is empty → error

Parsing is not possible. Reason: Top = S, LA = b: cell M[S, b] is empty → error. ∴ error, the string `b b c` is not in the language.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- FIRST(S): S → a A b gives { a }. Total { a }.
- FIRST(A): A → c B gives { c }. Total { c }.
- FIRST(B): B → d gives { d }. B → ε gives ε. Total { d, ε }.
- FOLLOW(S): start symbol → add $. Total { $ }.
- FOLLOW(A): in S → a A b: after A is b → add { b }. Total { b }.
- FOLLOW(B): in A → c B: nothing after B → add FOLLOW(A). Total { b }.

Lose marks by: writing only "error". Write the top, the LA and the empty cell.

Answer 2

WRITE THIS

Numbered rules: 1. S → A B 2. A → a A 3. A → ε 4. B → b B 5. B → c

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
S	{ a, b, c }	{ $ }
A	{ a, ε }	{ b, c }
B	{ b, c }	{ $ }

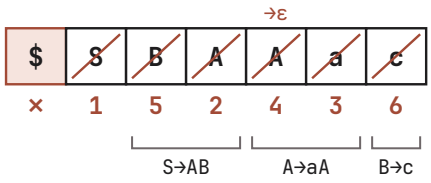
Step 2 · LL(1) parsing table

	a	b	c	$
S	S→AB	S→AB	S→AB	
A	A→aA	A→ε	A→ε	
B		B→bB	B→c	

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3 · Parse `a c d b` (horizontal stack)

Buffer: `a c d b $` LA ↑ starts at `a`



How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop S (LA = a): S→AB, push B A
2. pop A (LA = a): A→aA, push A a
3. match a, pop it, move LA → c
4. pop A (LA = c): A→ε, push nothing
5. pop B (LA = c): B→c, push c
6. match c, pop it, move LA → d
err. Top = $ but LA = d (input is left over) → error

Parsing is not possible. Reason: Top = $ but LA = d (input is left over) → error. ∴ error, the string `a c d b` is not in the language.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- FIRST(S): S → A B gives { a, b, c } (A can be ε → look at the next symbol). Total { a, b, c }.
- FIRST(A): A → a A gives { a }. A → ε gives ε. Total { a, ε }.
- FIRST(B): B → b B gives { b }. B → c gives { c }. Total { b, c }.
- FOLLOW(S): start symbol → add $. Total { $ }.
- FOLLOW(A): in S → A B: after A is B → add { b, c }. Total { b, c }.
- FOLLOW(B): in S → A B: nothing after B → add FOLLOW(S). Total { $ }.

Also note: d is not a terminal of this grammar at all.